

# Caution, Explosive!

Introduce of this quesition

Designed by ZHU Yueming

# What the meaning of this question?

Express the first test case

Input  
3 1 2  
0 1  
2 1  
Output  
7

| Step     | Pile  | Safety Box | Explain              |
|----------|-------|------------|----------------------|
| original | 0,1,2 | null       |                      |
| 1        | 0,2   | 1          | Take 1 from P to S   |
| 2        | 0,2   | 1          | Go back with nothing |
| 3        | 2     | 1,0        | Take 0 from P to S   |
| 4        | 1, 2  | 0          | Go back with 1.      |
| 5        | 1     | 0, 2       | Task 2 from P to S   |
| 6        | 1     | 0, 2       | Go back with nothing |
| 7        | null  | 0,1,2      | Take 1 from P to S   |

# How to design the **state** of materials and Llya

- The state consists of two parts:

**(1) Material position:** Represented by an n-bit binary number, each bit corresponds to a material.

- 0 indicates the material is in the pile
- 1 indicates the material is in the safe box.

For example, when n=5, state of material is 00101

| 4    | 3    | 2        | 1    | 0        |
|------|------|----------|------|----------|
| 0    | 0    | 1        | 0    | 1        |
| Pile | Pile | Safe box | Pile | Safe box |

**(2) Llya's position:** 0 indicates Llya is in the pile, and 1 indicates Llya is in the safe box.

- Initial state: (000...0, 0) (All materials are in the pile, Llya is in the pile)
- Target state: (111...1, 1) (All materials are in the safe box, Llya is in the safe box)

The initial state is:

| 4 | 3 | 2 | 1 | 0 | Llya |
|---|---|---|---|---|------|
| 0 | 0 | 0 | 0 | 0 | 0    |

# Using state to explain original Table

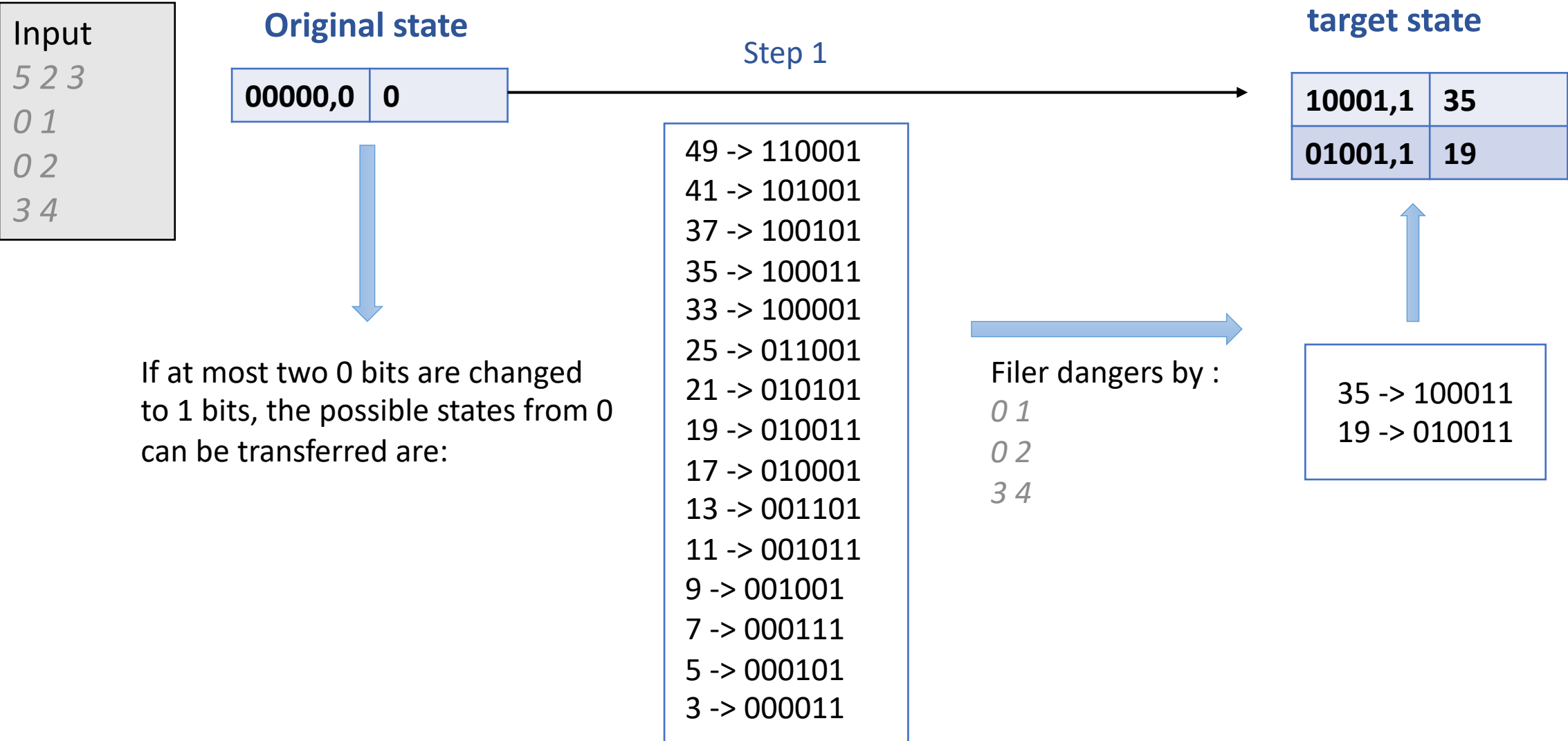
Input  
3 1 2  
0 1  
2 1  
Output  
7

| Step     | Pile  | Safety Box | Explain              | State |
|----------|-------|------------|----------------------|-------|
| original | 0,1,2 | null       |                      | 0000  |
| 1        | 0,2   | 1          | Take 1 from P to S   | 0101  |
| 2        | 0,2   | 1          | Go back with nothing | 0100  |
| 3        | 2     | 1,0        | Take 0 from P to S   | 0111  |
| 4        | 1, 2  | 0          | Go back with 1.      | 0010  |
| 5        | 1     | 0, 2       | Task 2 from P to S   | 1011  |
| 6        | 1     | 0, 2       | Go back with nothing | 1010  |
| 7        | null  | 0,1,2      | Take 1 from P to S   | 1111  |

# BFS design and State transition

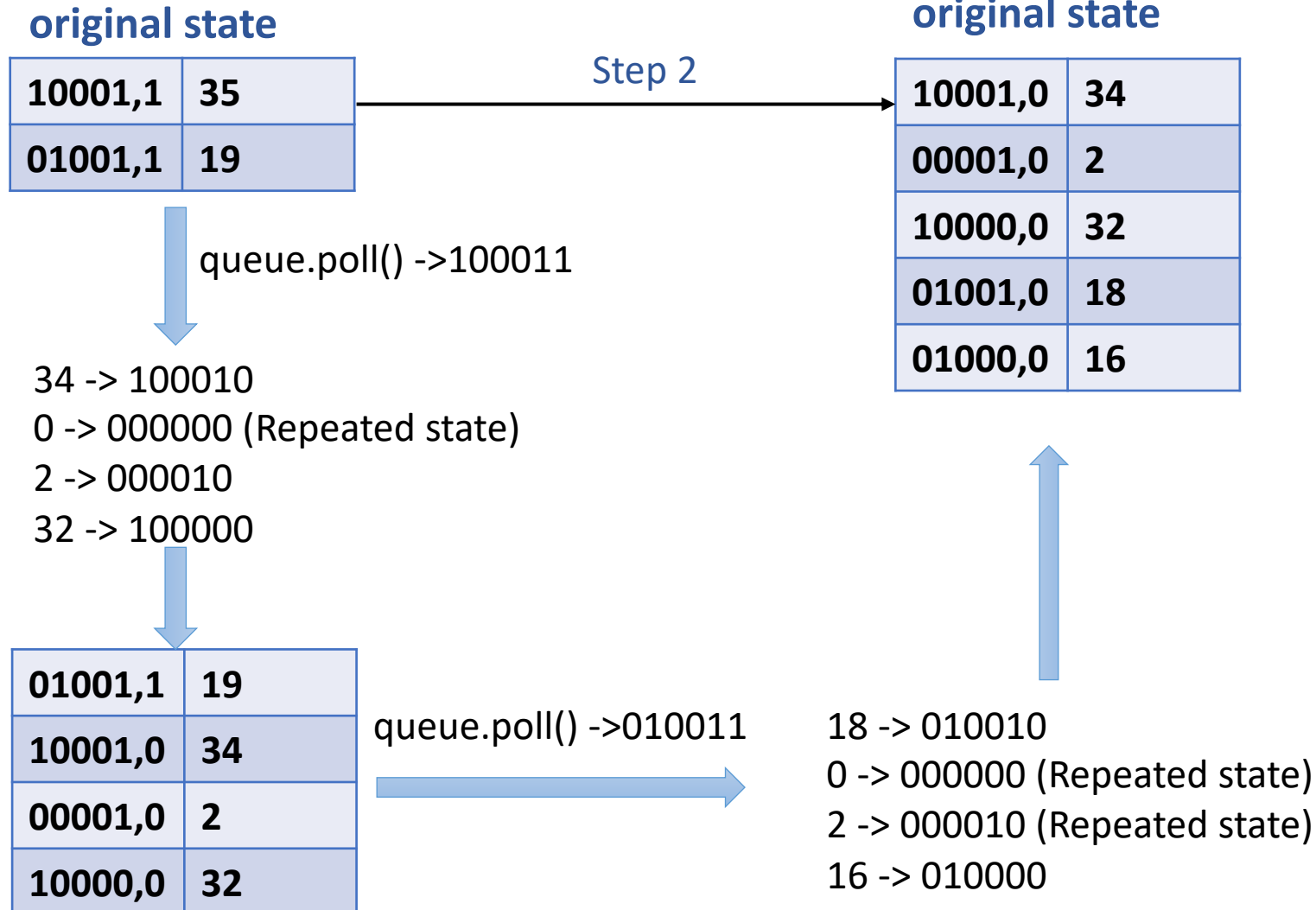
- Each state is a node in the graph.
- Each transition step involves an edge between the corresponding nodes.
  - **Ilya is in the pile (pos=0):** She can only transfer materials (0 bits) from the pile to the safe box.
  - **Ilya is in the safe box (pos=1):** She can only transfer materials (1 bit) from the safe box back to the pile.
- Browse using Breadth-First Search (BFS) to find the shortest path from the **initial state(00000)** to the **target state(11111)**; the path length is the number of transition steps.

# Example of Enumerate all states



# Example of Enumerate all states

|       |
|-------|
| Input |
| 5 2 3 |
| 0 1   |
| 0 2   |
| 3 4   |



# Design logic of Enumerate.

- How to store the state? For example, the state

|         |    |
|---------|----|
| 01001,1 | 19 |
|---------|----|

A String

"010011"

A int[] Array

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 1 | 1 |
|---|---|---|---|---|---|

A Class

|              |   |
|--------------|---|
| int material | 9 |
| int side     | 1 |

Only a Number

19





# Design logic of Enumerate.

- 1. Determine the set of movable materials:
  - Determine the changed bits, all unchanged bits are stayed the same.
- 2. Enumerate a subset of all movable materials:
- 3. Limit the quantity that can be moved:
  - Try `Integer.bitCount(9);`
- 4. Generate a new state:
  - Merge materials position with Ilya position
- 5. Filter out visited state
  - If you store the state using an interger number, it is easy to filter out duplicate state.
  - We can create an array like `state[2^20]` to store which state has been visited
- 6. Filter out dangerous situations:

# 1. Determine the set of **movable materials**:

- If Ilya is in the pile (side=0)
  - The movable material is all the 0 bits in the current state (materials in the pile)
  - For example, state = **010010** , material part = **01001**, The 5,3,2 material can be moved. So that the movable material is **10110**
  - **movable = ~state;**
  - **movable &= (1 << n) - 1; // only keep the first n digits**
- If Ilya is in the safe box (side=1)
  - The movable material is all the 1 bits in the current state (materials in the safe box)
  - For example, state = **010011** , material part = **01001**, The 4,1 material can be moved. So that the movable material is **01001**
  - **movable = state;**
  - **movable &= (1 << n) - 1; //only keep the first n digits**

## 2. Enumerate a subset of all movable materials:

- movable is the **largest number** that can currently be moved, so the remaining movable numbers will definitely be smaller than movable. Therefore, we find them one by one.
- For example: **movable = 01101 = 13**

```
13->01101 & 01101 = 01101
12->01100 & 01101 = 01100
11->01011 & 01101 = 01001
10->01010 & 01101 = 01000
9->01001 & 01101 = 01001
8->01000 & 01101 = 01000
7->00111 & 01101 = 00101
6->00110 & 01101 = 00100
5->00101 & 01101 = 00101
4->00100 & 01101 = 00100
3->00011 & 01101 = 00001
2->00010 & 01101 = 00000
1->00001 & 01101 = 00001
0->00000 & 01101 = 00000
```

Filter duplicate

```
01101
01100
01001
01000
00101
00100
00001
```

With this method, there is no need to loop 13 times;

$i = (i - 1) \& \text{movable}$  directly locates the values that satisfy the movement conditions

```
// the sample code of enumerate all subset
for (int i = movable; i > 0; i = (i - 1) & movable) {

}
```

# Bidirectional BFS Optimization

When  $n=20$ , the total number of states is  $2 * 2^{20} = 2,097,152$ , making ordinary BFS search inefficient.

- 1. Initialize two search queues:
  - Forward queue: Start searching from the initial state (000...0, 0)
  - Reverse queue: Start searching from the target state (111...1, 1)
- 2. Expand the two queues alternately:
  - Each time, choose the queue with fewer states to expand, reducing the search space.
  - During expansion, generate new states following the state enumeration approach and filter out dangerous states.
- 3. Check for queue intersection:
  - When a state is visited in both the forward and reverse queues, the total steps are the sum of forward steps and reverse steps.
  - Record the total steps for all intersecting states and take the minimum value as the answer.

# Improve Bidirectional BFS

- Due to the symmetry of the state space in this problem and the reversibility of operations, the reverse search can obtain the same results by inverting the states from the forward search.
- For example:
  - If we use  $n$  steps from 000000 to 110011, we will also use  $n$  steps to from 111111 to 001100 by reverse search.